

# Searching Algorithms

1

## Linear Search

- First we consider the **successful search**.
  - The best search time is  $\theta(1)$ , since the key will be stored in the first location/node.
  - In worst case, the key will be found in the last location/node. Thus, worst case search time  $\theta(n)$
  - On an average the list will be examined  $n/2$  times. Thus, average search time is  $\theta(n/2) = \theta(n)$
- In case of unsuccessful search all of the  $n$  elements are examined. Therefore, the running time for **unsuccessful search** is  $\theta(n)$

2

## Randomized Search

Suppose that array consists of *sorted* elements, and *index j* is used to search the array sequentially. Assume that the index is positioned at cell # 2. The subsequent steps of *randomized search* are illustrated in figures (a)-(d).

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 1  | 2  | 3  | 4  | 5   | 6   | 7   | 8   |
| 34 | 46 | 87 | 99 | 120 | 350 | 570 | 998 |

↑ *j*

Search key

(a) Initially, the loop index *j* points to cell 2. The key to be searched, 570, is located in cell 7

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 1  | 2  | 3  | 4  | 5   | 6   | 7   | 8   |
| 34 | 46 | 87 | 99 | 120 | 350 | 570 | 998 |

↑ *j*

Random number

(b) Random no 8 is generated. It is in the range 2-8 but element 998 is greater than search key, 570. The random number is ignored. Index *j* is incremented by 1 to continue the search

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 1  | 2  | 3  | 4  | 5   | 6   | 7   | 8   |
| 34 | 46 | 87 | 99 | 120 | 350 | 570 | 998 |

↑ *j*

Random number

(c) Random no 2 is generated. It is out of range 2-8, and ignored. The index *j* is incremented by 1. Search continues.

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 1  | 2  | 3  | 4  | 5   | 6   | 7   | 8   |
| 34 | 46 | 87 | 99 | 120 | 350 | 570 | 998 |

↑ *j*

Random number

(d) Random no 6 is generated. It is, within range 2-7 Index and key 360 less than 570. The index *j* is jumped to cell 6 by incrementing by 2

3

## Randomized Search

### Algorithm

The **Randomized Search** algorithm is used on a **presorted** Array. Randomized search is A modification of the linear search method. It proceeds as follows:

**Step # 1:** Starting with the first element, the array is scanned using an iterative loop.

**Step #2:** In each loop cycle a random number is generated in the range 1- *n*, where *n* is array size

**Step #3:** If the random number is greater than current value loop index, and the cell accessed by random number contains value less than or equal to key, then loop index is incremented to the random number ;otherwise, the loop index is incremented by 1

**Step#4:** Step#2 and Step #3 are repeated until the loop terminates.

4

# Binary Search Analysis

## Worst Case Running Time

In the **worst case**, search is continued until the subarray reduces to a single element..  
Let  $T(n)$  be the worst running time for searching an array of size  $n$ . Since the cost of partitioning is fixed, the following recurrence holds.  
:

$$T(1) = c$$

$$\underbrace{T(n)}_{\text{Searching array of size } n} = \underbrace{T(n/2)}_{\text{Searching subarray of size } n/2} + \underbrace{c}_{\text{Cost of computing mid point and comparing}}, \quad \text{when } n > 1$$

The recurrence can be solved by substitution method . It has the solution:

$$T(n) = \theta(\lg n)$$

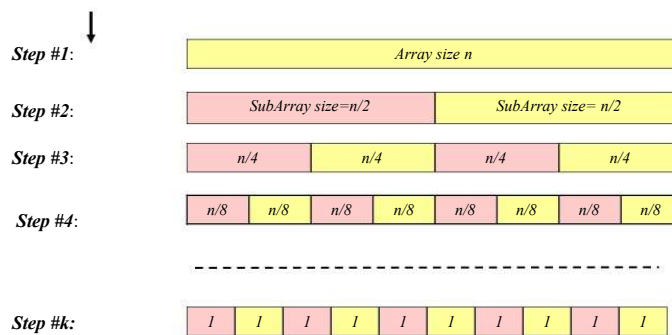
5

# Binary Search Analysis

## Expected Running Time-I

In binary search, at each unsuccessful search, the subarray size is **reduced by half**.

Assuming that the array size reduces to 1 after  **$k$  steps of partitioning**, then  $k = \lg(n)$



Partitioning of Array in Binary Search

6

## Binary Search Analysis

### Expected Running Time-II

In order to find the **expected running time**, we consider the probabilities of finding a given key at different steps.

If key is likely to in any of the  $n$  cells at the first step, the probability that key would be in any of the cells is  $1/n$ . Let  $c$  be unit cost of computing the index of mid element and comparing key

At the next step, the array size is reduced to  $n/2$ . Thus, the probability that the key is in any of the  $n/2$  cells is  $1/(n/2)=2/n$ . Also, the cost of second step is  $2c$ . Continuing, we can determine all the probabilities. the table summarizes the probabilities and the associated costs at all steps of binary search procedure.

| Step # | Subarray Size | Probability | Cost  |
|--------|---------------|-------------|-------|
| 1      | $n = n^0$     | $2^0/n$     | $c$   |
| 2      | $n/2 = n/2^1$ | $2^1/n$     | $2.c$ |
| 3      | $n/4 = n/2^2$ | $2^2/n$     | $3.c$ |
| ....   | .....         | .....       | ....  |
| i      | $n/2^{i-1}$   | $2^{i-1}/n$ | $i.c$ |
| --     | ----          | -----       |       |
| k      | 1             | 1           | $k.c$ |

If  $T(n)$  is the **expected search time** then

$$T(n) = c \cdot 2^0/n + 2c \cdot 2^1/n + 3c \cdot 2^2/n + \dots + kc$$

where  $k = \lg(n)$

7

## Binary Search Analysis

### Expected Running Time-III

From the preceding analysis we have,

$$T(n) = c \cdot 2^0/n + 2c \cdot 2^1/n + 3c \cdot 2^2/n + \dots + kc$$

where  $k = \lg(n)$

Or,

$$T(n) = (c/n) \sum_{i=1}^k i \cdot 2^{i-1}$$

Multiplying the right side with 2 and then dividing by 2, the summation is reduced to standard form:

$$T(n) = (c/2n) \sum_{i=1}^k i \cdot 2^i$$

The **arithmetic-geometric** series has the sum

$$\sum_{i=1}^k i \cdot 2^i/n = (k-1)2^{k+1} + 2$$

$$\begin{aligned} T(n) &= (c/2n) \cdot [(k-1)2^{k+1} + 2] = (c/2n) [\lg(n-1) \cdot 2^{\lg(n)+1} + 2] = (c/2n) [\lg(n-1) \cdot 2n + 2] \\ &= (c/2n) [2n \lg n - 2n + 2] = c \cdot (\lg n - 1 + 1/n) = \theta(\lg(n)) \end{aligned}$$

Thus, the **expected running time of binary search** algorithm is  $\theta(\lg n)$

8

## Comparison of Search Algorithms

Some typical number of comparisons for searching an array of one million items is as follows:

- *Linear search  $O(n)$ : 1,000,000 comparisons*
- *Randomized search  $O(\sqrt{n})$  :  $\sqrt{(1000000)} = 1,000$  comparisons*
- *Binary search  $O(\lg n)$  :  $\lg(1000000) = 6.\lg(10) \approx 19$  comparisons*

Binary search is **most efficient method**. However, both binary and randomized algorithms, presuppose the existence of **presorted** input. The sorting would take some additional running time  $O(n \lg n)$

Binary search cannot be applied to keys stored in a linked list.

9

## Binary Search Tree

### Operations

*Common operations performed on binary search tree are :*

**INSERT**- Add a new node with a given key

**SEARCH**- Search a node with a given key

**DELETE**- Delete a node with a given key

**MAXIMUM**- Find largest key in the tree or in a subtree

**MINIMUM**- Find smallest key in the tree or in a subtree

**SUCCESSOR**- Find a node with smallest key which larger than the key in given node

**TRAVERSE**- Traverse tree inorder, preorder, or postorder

10

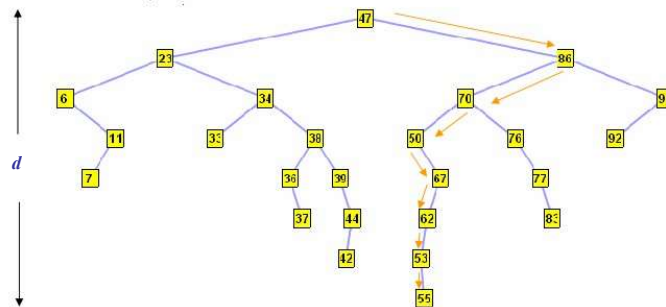
## Analysis of BST Search

### Worst Case

In **search** operation a given key is successively compared with the keys in child nodes. The total number of comparisons equals number of descents made. In worst case, descent is made down to the bottom level, as depicted in the figure. Therefore, **worst running time** for search operation is given by

$$T_{\text{search}} = O(d)$$

where  $d$  is BST depth.



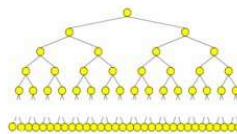
Search path in worst case

11

## Analysis of BST Search

### Best Case Scenario

In **best case scenario**, the BST has the **minimum depth**. This would be case when the tree is **full** or **nearly full**, as shown in the figure.



BST full tree

If  $n$  is the number of nodes in a full tree of depth  $d$ , then, since, the tree has  $1, 2, 4, \dots, 2^d$  nodes at different levels

$$n = 2^0 + 2^1 + 2^2 + \dots + 2^d = 2^{d+1} - 1$$

$$\text{or, } n+1 = 2^{d+1}$$

Taking logarithm, and rearranging:

$$d = \lg(n+1) - 1$$

The **best running time**,  $T_{\text{best}}(n)$ , for **search** operation is given by

$$\begin{aligned} T_{\text{best}}(n) &= O(d) = \lg(n+1) - 1 \\ &= O(\lg n) \end{aligned}$$

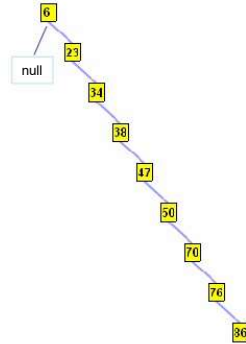
12

# Analysis of BST Search

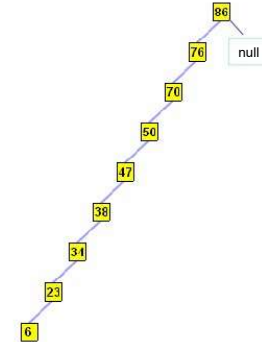
## Worst Case Scenario

In *worst case scenario*, the BST has the *maximum depth*. This would be case when the input keys are either *presorted* or *reverse sorted*, as shown in figures. In either case, tree degenerates into a *linked list* and tree depth equals the number of nodes in the tree, that is  $d=n$ . Thus, *worst case* running time,  $T_{\text{worst}}(n)$ , for searching BST is given by

$$T_{\text{worst}}(n) = O(d) = O(n)$$



(a) BST with presorted keys



(b) BST with reverse sorted keys